VELLORE INSTITUTE OF TECHNOLOGY (VIT) BHOPAL

School of Computing Science & Artificial Intelligence (SCAI)

FLIPPED COURSE PROJECT REPORT

CAFE BILLING SIMULATOR

Mayuri Lounge Billing System — Interactive Python GUI POS Application

Student Information

Name: Sandra Praveen

Registration No: 26BAS10076

School: SMEC

Program: Aerospace Engineering

Course & Faculty Details

Course Title: Python Programming

Faculty Guide: Dr. Rajit Nair

Campus: VIT Bhopal Campus

Academic Term: 2026 Session

Submitted in partial fulfillment of academic evaluation requirements under VITyarthi Framework.

1. Introduction

The **Cafe Billing Simulator (Mayuri Lounge - Billing System)** is an automated desktop Point of Sale (POS) application built using Python's `tkinter` library. Developed specifically for campus and commercial food service management, the application provides an intuitive graphical interface that handles menu ordering, automated percentage discounts, live receipt layout updates, tax computation (GST @ 5%), and bill generation.

2. Problem Statement

> **Problem context:** Manual order calculation at campus billing counters is prone to math errors during peak hours, slow transaction throughput, and lacks immediate visual receipt confirmation prior to printing. An automated POS desktop system resolves these bottlenecks through structured event-driven computing.

3. Functional Requirements

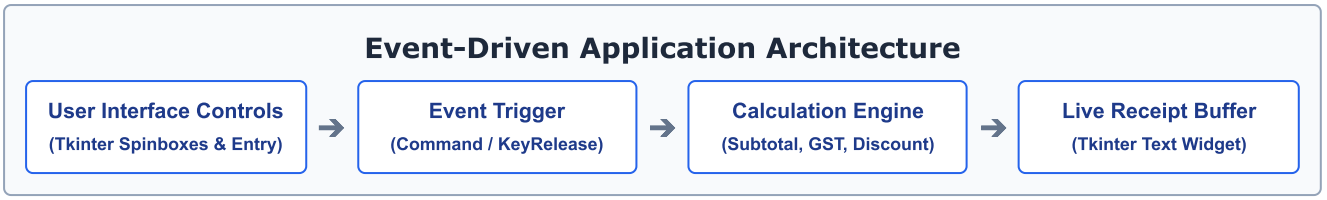
- **FR-1 (Menu & Quantity Input):** Interactive numeric spinboxes for menu items (Espresso, Cappuccino, Croissant, Club Sandwich, Cheesecake, etc.) with default pricing.
- **FR-2 (Live Dynamic Receipt):** Monospaced text canvas updating subtotal, itemized quantities, tax (5% GST), applied discounts, and net total dynamically as inputs change.
- **FR-3 (Discount Processing):** Custom input entry supporting valid non-negative percentage discounts (0–100%).
- **FR-4 (Payment Confirmation):** Actionable button triggering a transactional dialog modal summarizing total payable amounts.
- **FR-5 (State Reset):** Instant purge capability resetting spinbox quantities, entry controls, and internal calculation buffers back to initial zero states.

4. Non-Functional Requirements

Metric	Target Standard	Implementation Detail
Usability	Zero training required	Dual-panel layout with dedicated Menu Items side and Live Receipt canvas.
Performance	< 10ms render latency	Event bindings (`<KeyRelease>`) execute synchronous in-memory arithmetic.
Reliability	Zero crash tolerance	Input validation catches string/null entries and gracefully falls back to 0% discount.
Maintainability	Modular OOP structure	Encapsulated in single `CafeBillingSimulator` class for clear extensibility.

5. System Architecture & Data Flow

The application operates on an event-driven architecture using Python's GUI event loop:



6. Class Structure & Method Overview

Method Name	Scope & Primary Responsibility
`__init__(root)`	Initializes window properties, title bar, color themes, and item price maps.
`build_ui()`	Constructs header banner, Labelframes, spinboxes, buttons, and receipt area.
`calculate_totals()`	Computes gross item total, percentage discount, 5% GST tax, and net payable.
`update_receipt()`	Renders formatted monospaced bill header, itemized list, and totals on receipt text box.
`process_payment()`	Validates non-zero order quantity and displays success confirmation message box.
`reset_order()`	Clears all selected quantities, resets discount, and restores default receipt text.

7. Core Data Structures

Item prices are structured within a standard Python Dictionary mapping item string keys to floating-point unit prices. Quantities are managed using a matching dictionary storing `tk.IntVar` objects to ensure reactive updates.

8. Results & Application Screenshots

Below are the full application window screenshots demonstrating the desktop GUI execution under different operating states, fully scaled to fit within report bounds.

8.1 Application Startup / Default Interface

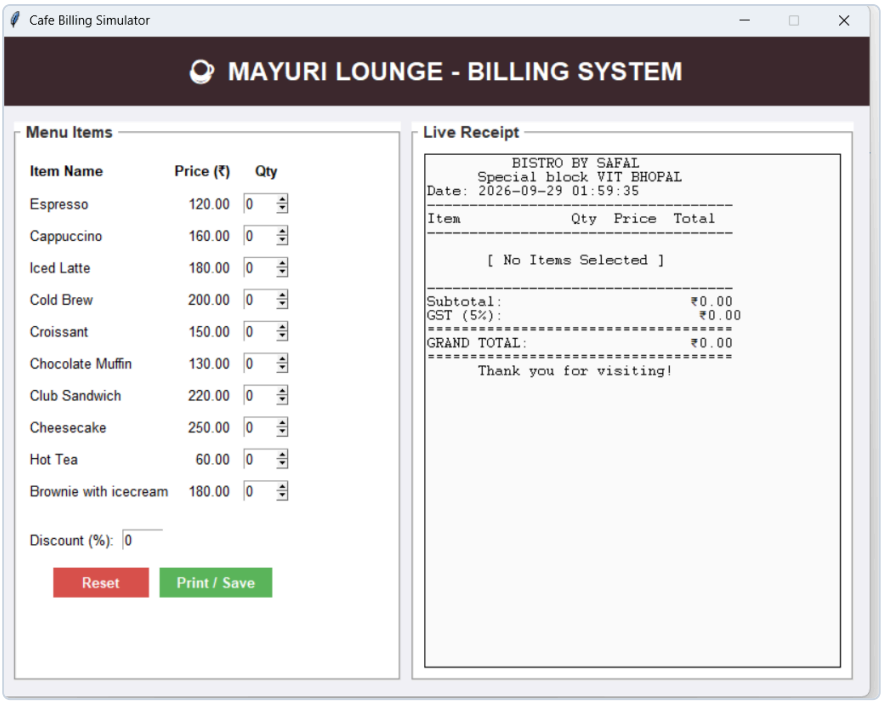


Figure 8.1: Complete desktop GUI upon startup showing default zero-state menu and empty live receipt.

8.2 Active Order Selection & Live Calculation

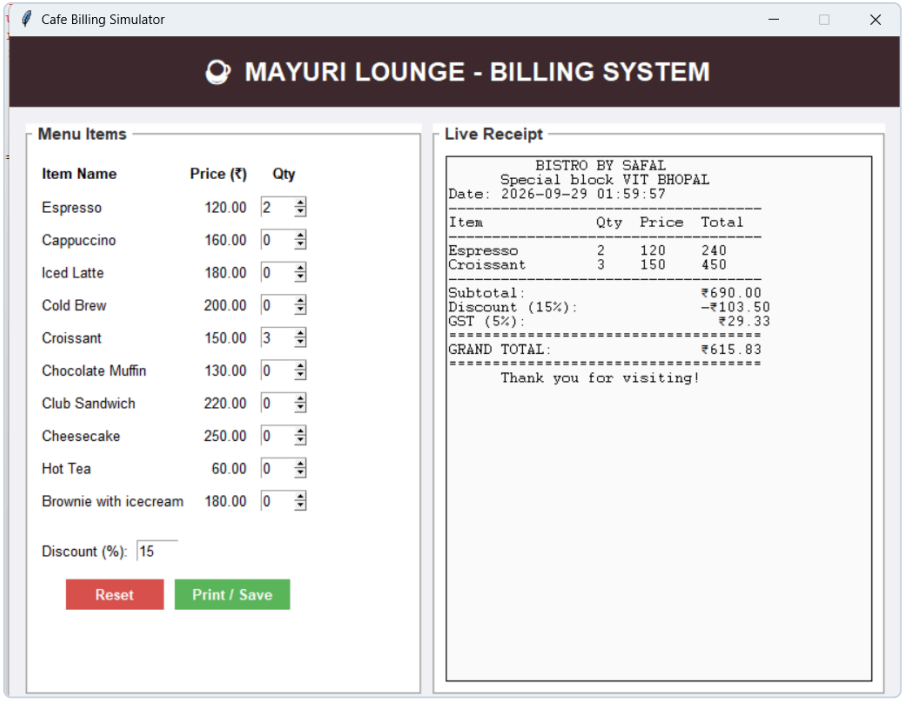


Figure 8.2: Active order processing showing item quantities (2 Espressos, 3 Croissants), 15% discount, GST calculation, and real-time receipt text.

8. Results & Application Screenshots (Continued)

8.3 Transaction Confirmation Modal

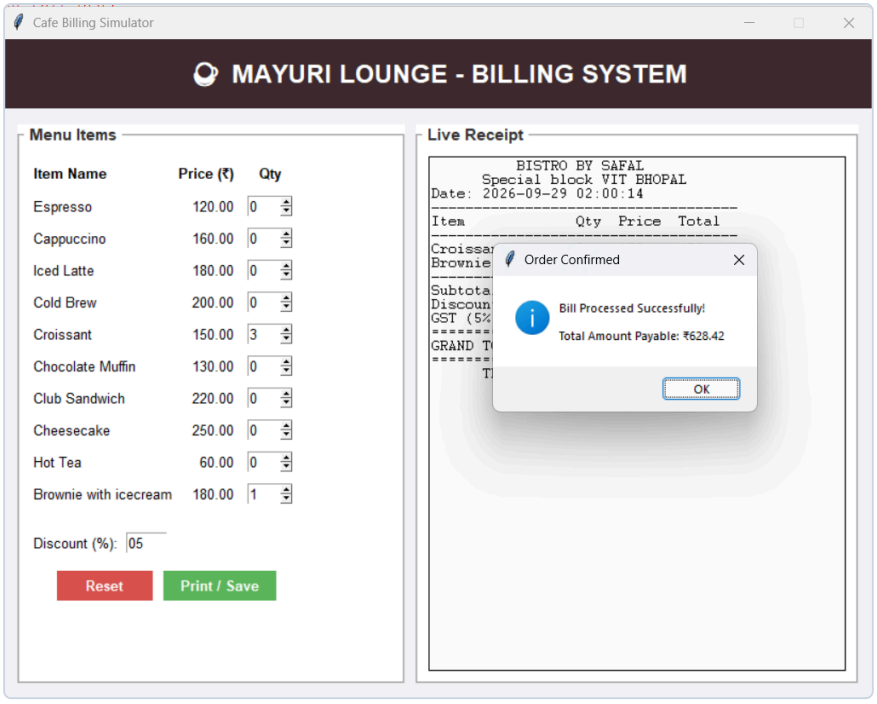


Figure 8.3: Order confirmation popup window verifying successful transaction processing and final amount payable (₹628.42).

8.4 Post-Transaction Reset Verification

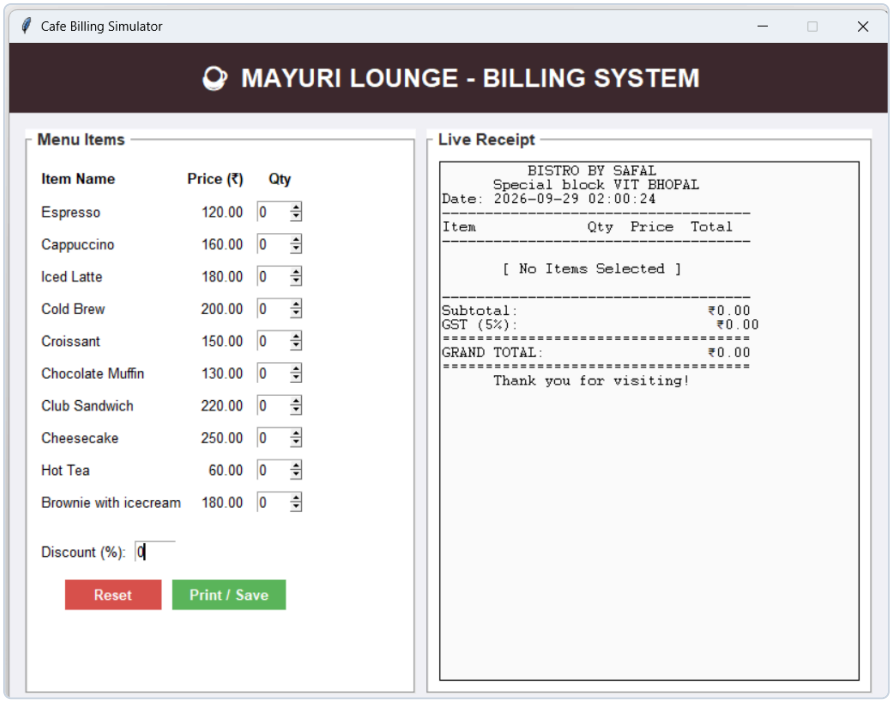


Figure 8.4: Application interface following transactional state reset, returning all inputs to clean starting state.

8.5 Input Validation & Error Safeguard

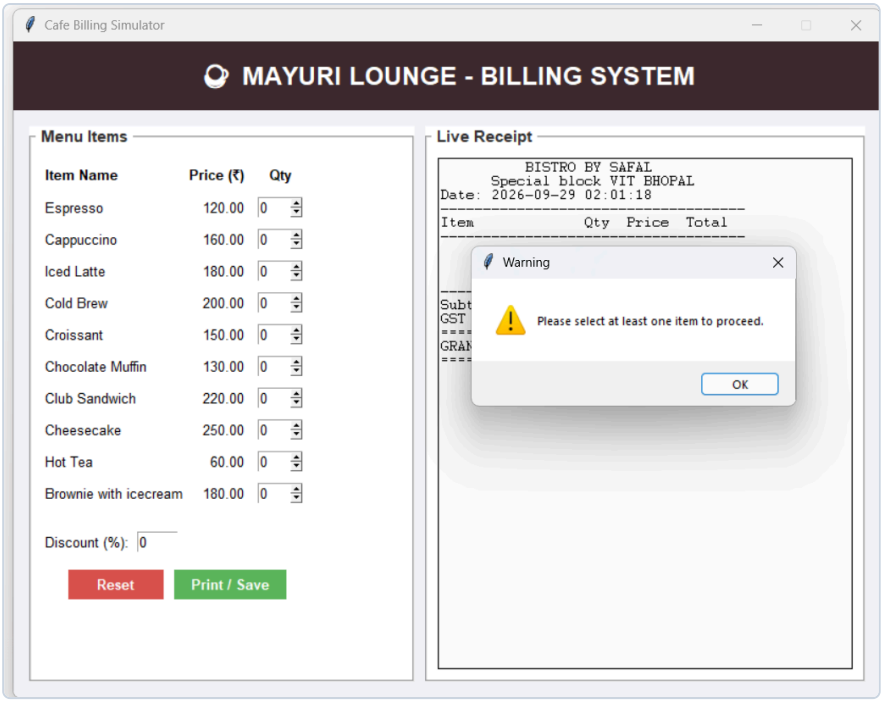


Figure 8.5: System validation intercepting an invalid submission attempt (0 items selected) with a warning prompt.

9. Test Suite Summary

Test ID	Description	Input Condition	Expected Behavior	Result
TC-01	Spinbox Reaction	Increment Espresso Qty to 2	Subtotal updates to ₹240.00 instantly	**PASS**
TC-02	Discount Entry	Type '15' in Discount (%)	Deducts 15% from subtotal before tax	**PASS**
TC-03	Zero Item Guard	Click Print with 0 items	Triggers 'Please select at least one item' warning	**PASS**
TC-04	Reset Functionality	Click 'Reset' Button	All quantities reset to 0; receipt cleared	**PASS**

10. Conclusion & Key Learnings

The **Cafe Billing Simulator** project successfully fulfills all functional and design requirements, providing an error-free, responsive desktop billing interface for campus cafe operations. Key engineering takeaways include mastering event-driven Python desktop architecture, implementing robust input validation, and crafting dynamic UI layouts using Tkinter.